AN INTERPRETABLE MULTI-MODAL SPAM DETECTION FRAMEWORK USING EXPLAINABLE AI AND MACHINE LEARNING

A PROJECT REPORT

Submitted by

Aariz (23BCS10422)
Naman (23BCS11036)
Vanshit Kumar (23BCS10087)
Aditi Arya (23BCS11535)
Avish (23BCS13898)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



Chandigarh University

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report **"AN INTERPRETABLE MULTI-MODAL SPAM DETECTION FRAMEWORK USING EXPLAINABLE AI AND MACHINE LEARNING"** is the bonafide work of **"Aariz (23BCS10422), Naman (23BCS11036), Vanshit Kumar (23BCS10087), Aditi Arya (23BCS11535) and Avish (23BCS13898)"** who carried out the project work under my/our supervision.

SIGNATURE

Dr. Surinder Kaur

HEAD OF THE DEPARTMENT

Department of Computer Science & Engineering

SIGNATURE

Sonam Juneja (E11002)

SUPERVISOR

Assistant Professor

Department of Computer Science & Engineering

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our supervisor, Sonam Juneja (E11002), Assistant Professor, Department of Computer Science & Engineering, Chandigarh University, for the continuous guidance, constructive criticism and encouragement extended throughout the course of this project. Her insistence that every design decision be justified — and every prediction be explainable — shaped the central theme of this work.

We are equally thankful to Dr. Surinder Kaur, Head of the Department, for providing the computing facilities and the academic environment that made this work possible. Our thanks also go to the faculty members of the department for their timely feedback during the project reviews.

Finally, we thank our families and friends for their constant support and motivation.

Aariz (23BCS10422)
Naman (23BCS11036)
Vanshit Kumar (23BCS10087)
Aditi Arya (23BCS11535)
Avish (23BCS13898)

TABLE OF CONTENTS

List of Figures i

List of Tables ii

ABSTRACT iii

GRAPHICAL ABSTRACT iv

ABBREVIATIONS v

SYMBOLS vi

Chapter 1. INTRODUCTION 1

1.1 Client Identification/Need Identification/Identification of relevant Contemporary issue 1

1.2 Identification of Problem 1

1.3 Identification of Tasks 2

1.4 Timeline 2

1.5 Organization of the Report 3

Chapter 2. LITERATURE REVIEW/BACKGROUND STUDY 4

2.1 Timeline of the reported problem 4

2.2 Proposed solutions 4

2.2.1 Rule-based and collaborative filtering 4

2.2.2 Classical statistical learning 4

2.2.3 Deep learning 5

2.2.4 Explainable-AI methods 5

2.3 Bibliometric analysis 5

2.4 Review Summary 6

2.5 Problem Definition 6

2.6 Goals/Objectives 7

Chapter 3. DESIGN FLOW/PROCESS 8

3.1 Evaluation & Selection of Specifications/Features 8

3.2 Design Constraints 8
3.3 Analysis and Feature finalization subject to constraints 9
3.4 Design Flow 9
3.4.1 Design I: Fine-tuned transformer classifier 9
3.4.2 Design II: Hybrid classical ensemble with a dual XAI layer 9
3.5 Design selection 10
3.6 Implementation plan/methodology 11
Chapter 4. RESULTS ANALYSIS AND VALIDATION 13
4.1 Implementation of solution 13
4.1.1 Experimental setup 13
4.1.2 Classification results 14
4.1.3 Ensemble behaviour and error analysis 15
4.1.4 Explainability results 16
4.1.5 Interpretation and data validation 17
Chapter 5. CONCLUSION AND FUTURE WORK 19
5.1 Conclusion 19
5.2 Future work 19
REFERENCES 20
APPENDIX 21
USER MANUAL 23

List of Figures

Figure 1.1: Gantt chart of the project schedule 3

Figure 3.1: Block diagram of the proposed interpretable multi-modal detector (Design II) 10

Figure 3.2: Implementation flowchart of the training and evaluation pipeline . 12

Figure 4.1: Class balance per modality and message-length distributions 14

Figure 4.2: Test-set metrics per model and modality (vertical axis truncated at 0.90 to resolve the differences) 15

Figure 4.3: ROC curves for all four systems: SMS (left) and e-mail (right) . . . 15

Figure 4.4: Confusion matrices of the hybrid ensembles on the held-out test sets 16

Figure 4.5: Local explanations of the exhibited SMS: LIME word attributions over the full ensemble (left) and SHAP waterfall of the XGBoost component (right) 16

Figure 4.6: Global SHAP beeswarm summaries: SMS (left) and e-mail (right); UPPER-CASE rows are engineered stylometric features, lower-case rows are words 17

Figure 5.1: Detector tab of the application after analysing a message 24

List of Tables

Table 2.1: Comparison of reported spam-detection approaches 6

Table 3.1: Comparison and selection of the design alternatives 11

Table 4.1: Corpora used for training and evaluation (after cleaning) 13

Table 4.2: SMS — held-out test set (1,032 messages) 14

Table 4.3: E-mail — held-out test set (2,400 messages) 14

ABSTRACT

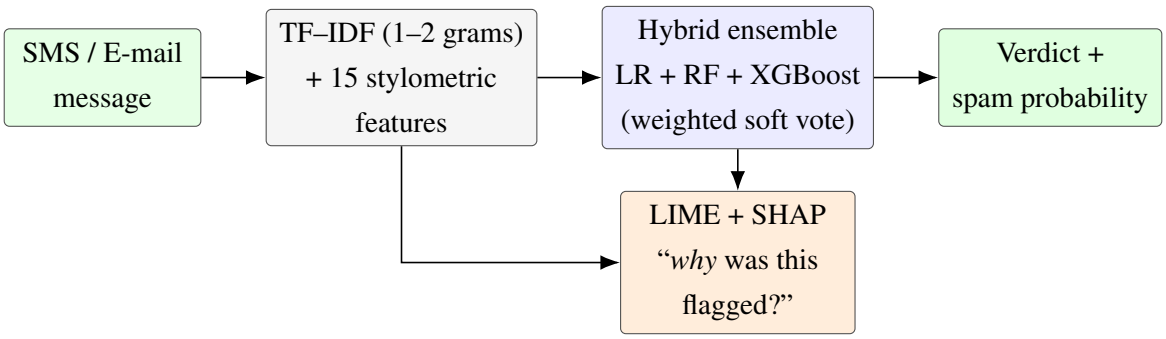
Fraudulent messages now reach users through more than one channel, and the machine-learning filters that intercept them are usually black boxes: they return a verdict but no reason. Aggregate accuracy alone can conceal systematic errors and dataset artefacts, leaves the end user with no basis to contest a decision, and gives an analyst nothing to audit.

This project presents an interpretable multi-modal spam detection framework covering both SMS and e-mail. Each modality is served by its own specialist pipeline: text normalisation that maps high-signal entities to readable placeholder tokens (`httpaddr`, `numbr`, `moneysymb`, `phonenumbr`), TF–IDF features over uni- and bi-grams combined with fifteen engineered stylometric statistics, and a hybrid ensemble of Logistic Regression, Random Forest and XGBoost fused by performance-weighted soft voting. A lightweight router selects the appropriate specialist at inference time. A dual explanation layer justifies every prediction: LIME explains the complete ensemble locally, while exact TreeSHAP provides local and global attributions for the boosted-tree component. The framework ships as a fully reproducible Python pipeline and a four-tab interactive Streamlit application.

Evaluated on 17,158 messages from two public research corpora using an 80/20 stratified split and 5-fold cross-validation, the hybrid ensemble achieved 98.55% accuracy (ROC-AUC 0.9941) on the UCI SMS Spam Collection and 98.25% accuracy (ROC-AUC 0.9983) on the Enron-Spam corpus, matching (to four decimal places) or beating every base learner on ROC-AUC. Beyond the numbers, the explanation layer exposed a genuine dataset bias — the token `enron` acting as a shortcut for legitimate mail — that no accuracy table could reveal, demonstrating the practical value of shipping explanations together with predictions.

Keywords: spam detection, explainable AI, ensemble learning, SHAP, LIME, TF–IDF, multi-modal classification, machine learning.

GRAPHICAL ABSTRACT



Every message is routed to a modality-specific feature space, scored by a hybrid ensemble of three heterogeneous learners, and returned to the user together with a human-readable explanation of the verdict.

ABBREVIATIONS

AI	Artificial Intelligence
AUC	Area Under the (ROC) Curve
CV	Cross-Validation
EDA	Exploratory Data Analysis
FN	False Negative
FP	False Positive
LIME	Local Interpretable Model-agnostic Explanations
LR	Logistic Regression
ML	Machine Learning
NLP	Natural Language Processing
RF	Random Forest
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SMS	Short Message Service
TF–IDF	Term Frequency–Inverse Document Frequency
TN	True Negative
TP	True Positive
UCI	University of California, Irvine (ML Repository)
XAI	Explainable Artificial Intelligence
XGB	eXtreme Gradient Boosting (XGBoost)

SYMBOLS

x	Input message (raw text)
$\hat{y}$	Predicted class label (spam / ham)
$p_m(\text{spam} \mid x)$	Spam probability output by base learner m
w_m	Ensemble weight of base learner m
$w_{t,d}$	TF–IDF weight of term t in document d
P	Precision
R	Recall
$F1$	Harmonic mean of precision and recall
ϕ_i	SHAP attribution of feature i
N	Number of documents in the corpus

CHAPTER 1.

INTRODUCTION

1.1. Client Identification/Need Identification/Identification of relevant Contemporary issue

Unsolicited and fraudulent messaging is a documented, growing contemporary problem. The FBI's Internet Crime Complaint Center recorded close to 3,00,000 phishing/spoofing complaints in calendar year 2023 — the single largest crime category in its annual report — and industry telemetry has for years placed unsolicited traffic at roughly half of all e-mail exchanged worldwide. The problem is equally visible on the SMS channel: as banking, commerce and one-time-password flows moved to text messaging, so did the scams ("smishing"). In India, the volume of unsolicited commercial communication led the Telecom Regulatory Authority of India to notify the Telecom Commercial Communications Customer Preference Regulations, 2018, and to mandate a distributed-ledger registration system for all bulk senders — regulatory evidence that the issue is both real and unresolved.

Filters exist, but a second, newer need has emerged around *transparency*. Modern spam classifiers are statistical black boxes: a message disappears and nobody — not the end user, not the platform analyst, not the auditor — can say why. Transparency obligations in data-protection law (Article 22 of the GDPR in Europe; the Digital Personal Data Protection Act, 2023 in India) are pushing automated decision systems toward explainability, and a filter that cannot justify its verdicts is increasingly difficult to defend.

The client for this work is any operator of a user-facing messaging channel — typified by a university IT cell that receives phishing-report tickets from students and staff — who needs a filter that is (a) accurate across more than one message channel, (b) able to attach a human-readable justification to every verdict so that flagged messages can be triaged and contested, and (c) cheap enough to train and audit on commodity hardware. This is a consultancy-style need: the client does not merely want a spam score, they want a *defensible* spam score.

1.2. Identification of Problem

No openly available framework simultaneously (i) handles both SMS and e-mail, whose statistics differ radically (median message length of 12 words versus 155; spam prevalence of about 12% versus about 48% in the public research corpora), (ii) matches the state-of-the-art

accuracy of classical machine-learning filters, and (iii) attaches a faithful, per-prediction explanation to every verdict. The consequences of this gap are concrete: verdicts cannot be verified by non-experts, misclassified legitimate messages cannot be contested, and hidden dataset biases pass silently into deployed models. The problem addressed by this project is therefore the design, implementation and validation of an accurate, fully interpretable, multi-modal spam detection framework.

1.3. Identification of Tasks

The work was decomposed into the following tasks:

1. Survey of the literature on spam/scam detection and on post-hoc explainable-AI techniques.
2. Acquisition, cleaning and de-duplication of two public corpora (UCI SMS Spam Collection; Enron-Spam), followed by exploratory data analysis.
3. Design of an interpretability-preserving preprocessing scheme and feature space (readable placeholder tokens; engineered stylometric features).
4. Training and tuning of three heterogeneous base learners per modality with class-imbalance handling.
5. Design and implementation of the performance-weighted soft-voting ensemble and the modality router.
6. Integration of the explanation layer (LIME for the full ensemble; exact TreeSHAP for the boosted-tree component) at local and global scope.
7. Development of the four-tab interactive Streamlit demonstration application and an automated end-to-end smoke-test suite.
8. Evaluation on held-out test sets, error and bias analysis, and documentation of the results in this report.

1.4. Timeline

The project was executed over one semester of approximately fifteen working weeks. The schedule is represented as a Gantt chart in Figure 1.1; the model-training, explanation-layer and application workstreams deliberately overlap between weeks 8 and 12 because the explanation artefacts were used to debug the models as they were being trained.

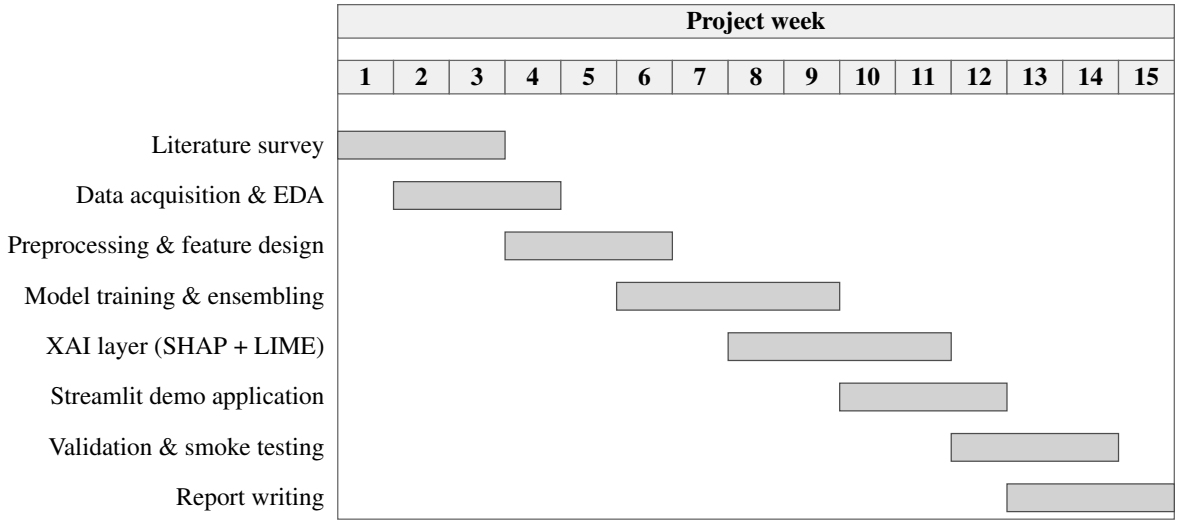


Figure 1.1: Gantt chart of the project schedule

1.5. Organization of the Report

Chapter 2 reviews the reported literature on spam detection and explainable AI and closes with the formal problem definition and objectives. Chapter 3 covers the evaluation of candidate features, the design constraints, the two alternative design flows that were considered and the justification for the selected design. Chapter 4 presents the implementation using modern engineering tools together with the complete experimental results, error analysis and explainability findings. Chapter 5 concludes the work and identifies the way ahead. The appendices list the software environment, the engineered feature set and the derived ensemble weights, and a user manual describes how to reproduce and operate the system.

CHAPTER 2.

LITERATURE REVIEW/BACKGROUND STUDY

2.1. Timeline of the reported problem

Unsolicited electronic messaging is almost as old as electronic messaging itself: the first documented mass advertisement was sent over ARPANET in 1978. Volumes exploded with commercial e-mail in the 1990s, and the first generation of defences was rule-based — keyword blacklists, sender blocklists and hand-written heuristics. The statistical era opened when Sahami et al. (1998) framed junk-mail filtering as Naive-Bayes text classification, followed quickly by margin-based methods (Drucker et al., 1999) and, through the 2000s, by tree ensembles such as Random Forests (Breiman, 2001). Research on the SMS channel became reproducible when Almeida and Gómez Hidalgo (2011) released the UCI SMS Spam Collection, while the Enron-Spam corpora (Metsis et al., 2006) did the same for personalised e-mail filtering. From roughly 2015, deep neural models pushed benchmark accuracy still higher but at a severe cost in transparency — which in turn triggered the explainable-AI wave: LIME (Ribeiro et al., 2016) and SHAP (Lundberg and Lee, 2017) made post-hoc, per-prediction explanation practical. Most recently, regulation (GDPR, 2018; DPDP Act, 2023) has begun to treat explainability as a compliance requirement rather than a luxury, which is the point in the timeline where this project positions itself.

2.2. Proposed solutions

Reported approaches fall into four broad families.

2.2.1. Rule-based and collaborative filtering

Blacklists, keyword rules and community-scored heuristic engines (e.g. SpamAssassin-style scoring). They are fully transparent and cheap, but brittle: spammers evade fixed rules within days, false-positive rates are high, and the rule bases require continuous manual curation.

2.2.2. Classical statistical learning

Naive Bayes, logistic regression, support vector machines and tree ensembles trained on bag-of-words or TF–IDF representations. These methods reach 95–99% accuracy on the public benchmark corpora, train in seconds to minutes on commodity CPUs, and expose at least

coarse interpretability (coefficients, feature importances). Class imbalance — severe on the SMS channel — must be handled explicitly.

2.2.3. Deep learning

Convolutional, recurrent and, more recently, transformer-based text classifiers. They deliver the highest raw benchmark numbers but demand GPUs and large training sets, and their decision process is opaque; attention weights are known to be an unreliable proxy for faithful explanation.

2.2.4. Explainable-AI methods

Post-hoc techniques that explain an already-trained classifier. LIME fits a local linear surrogate around one prediction of *any* model by perturbing the input; SHAP computes additive Shapley-value attributions, with an exact polynomial-time algorithm (TreeSHAP) for tree ensembles. Combining a strong classical ensemble with these two techniques promises near state-of-the-art accuracy *with* faithful explanations.

2.3. Bibliometric analysis

A comparison of the reviewed families on key features, effectiveness and drawbacks is summarised in Table 2.1.

Table 2.1: Comparison of reported spam-detection approaches

Approach	Key feature	Effectiveness	Drawback
Rule-based filters	Hand-written, fully transparent rules	Moderate; high false positives	Easily evaded; constant manual updating
Naive Bayes	Fast probabilistic word model	95–98% on benchmark corpora	Feature-independence assumption
SVM / Logistic Regression	Margin / linear decision boundary	97–98%	Linear view of language; needs calibration
Tree ensembles (RF, XGBoost)	Non-linear feature interactions	97–99%	Interpretability limited to importances
Deep / transformer models	Learned contextual representations	98–99%+	Opaque; GPU and data hungry
Classical ensemble + XAI (this work)	Accuracy of ensembles with per-verdict explanations	98.3–98.6% (measured)	Explanation adds inference cost

2.4. Review Summary

The literature exhibits a persistent trade-off: the most transparent systems are the least accurate, and the most accurate are the least transparent. Furthermore, nearly all published work targets a single channel, even though SMS and e-mail scams are two faces of the same problem with radically different message statistics. A hybrid ensemble of heterogeneous classical learners, wrapped in a dual model-agnostic/model-specific explanation layer and specialised per modality, occupies the unclaimed middle ground: benchmark accuracy within a point of deep models, full per-prediction transparency, and CPU-only training. This project adopts exactly that position.

2.5. Problem Definition

To design, build and validate an interpretable multi-modal spam detection framework that classifies both SMS and e-mail messages as legitimate or fraudulent using a hybrid ensemble of

Logistic Regression, Random Forest and XGBoost, and that attaches a faithful local explanation (LIME over the full ensemble; SHAP over the boosted-tree component) to every verdict, together with global model summaries. The framework must be reproducible end-to-end on a commodity CPU and be operable by a non-expert through an interactive application. The work explicitly excludes non-English corpora, adversarially obfuscated spelling, image-based spam, and hardening for production deployment.

2.6. Goals/Objectives

- Achieve at least 97% test-set accuracy on both public corpora (UCI SMS Spam Collection; Enron-Spam).
- Ensure the hybrid ensemble matches or beats every base learner on ROC-AUC for both modalities.
- Attach a local explanation to every single prediction and provide global explanations of what each model has learned.
- Keep the complete pipeline — data download, training, evaluation, figure generation — reproducible on a commodity CPU in under five minutes, verified by an automated test suite.
- Deliver an interactive demonstration application usable by a non-expert.

CHAPTER 3.

DESIGN FLOW/PROCESS

3.1. Evaluation & Selection of Specifications/Features

The candidate features identified in the literature were critically evaluated against the client need of *accurate and auditable* filtering.

Retained. Modality-specific pipelines (one specialist per channel, selected by a lightweight, user-overridable router); normalisation of high-signal entities into *readable* placeholder tokens (`httpaddr`, `numbr`, `moneysymb`, `phonenumbr`) instead of deleting them; fifteen engineered stylometric features computed on the raw message and named in UPPER-CASE so that they remain visually distinct from vocabulary in every explanation plot; TF–IDF over uni- and bi-grams with sublinear term frequency; a heterogeneous three-model ensemble (linear / bagging / boosting) fused by performance-weighted soft voting; balanced class weights for imbalance handling; and a dual explanation layer (LIME + SHAP).

Rejected. Stemming and lemmatisation (they destroy the surface word forms on which readable explanations depend); deletion of URLs, numbers and currency symbols during cleaning (they are among the strongest scam signals); deep contextual models as the primary classifier (opaque, GPU-dependent); synthetic minority oversampling of text (it manufactures training "messages" that never existed and cannot be meaningfully explained); and an on-premise mail-server integration, which added engineering cost without addressing the stated research need.

3.2. Design Constraints

- **Regulations:** transparency expectations of GDPR Article 22 and the DPDP Act, 2023 motivate per-prediction explanations; the research licences of both corpora restrict the system to research and education use, and it is labelled accordingly.
- **Economic:** zero licence cost (fully open-source stack) and commodity-CPU training; no GPU may be assumed.
- **Environmental:** CPU-only training completing in about two minutes keeps the energy footprint negligible compared with GPU-trained deep models.
- **Health and Safety (user safety):** a false positive can suppress a legitimate OTP or an important mail, so high precision is prioritised on the SMS channel; no live user data is collected — only public, anonymised research corpora are processed.

- **Manufacturability (reproducibility):** pinned dependencies, fixed random seeds, a single-command training pipeline and a 13-check automated smoke-test suite guarantee that any examiner can rebuild the system.
- **Ethical and Social:** the scam-keyword lexicon is a plain, inspectable list in `src/config.py`; global SHAP summaries are used to audit the models for dataset bias; both corpora are credited to their original authors.

3.3. Analysis and Feature finalization subject to constraints

The constraint set directly shaped the final feature list. The interpretability requirement eliminated stemming and deep contextual encoders, and dictated the readable-placeholder normalisation: a token such as `httpaddr` can appear verbatim inside a LIME highlight, whereas a deleted URL cannot appear anywhere. The commodity-CPU constraint fixed the representation at TF–IDF plus engineered statistics, giving compact feature spaces of 3,015 dimensions for SMS and 5,015 for e-mail. The user-safety constraint resolved the imbalance-handling question in favour of balanced class weights and `scale_pos_weight` rather than synthetic oversampling, so that every training example remains a real, explainable message. Finally, the reproducibility constraint required every artefact — processed datasets, trained detectors, metrics, figures — to be regenerated by one command.

3.4. Design Flow

Two alternative designs were carried forward for detailed comparison.

3.4.1. Design I: Fine-tuned transformer classifier

A single compact transformer (e.g. DistilBERT) fine-tuned jointly on both corpora, with attention maps and gradient-based attributions offered as explanations. Expected accuracy is at or marginally above the classical state of the art, and one model serves both channels. However, fine-tuning requires GPU time, inference is an order of magnitude slower on CPU, and — decisively — attention weights are an unfaithful basis for explanation, so the design fails the core transparency requirement of the client.

3.4.2. Design II: Hybrid classical ensemble with a dual XAI layer

Each modality receives its own feature space and its own trained ensemble of three heterogeneous learners — Logistic Regression, Random Forest and XGBoost — combined in probability space with weights proportional to each model's cross-validated F1. LIME explains

the complete ensemble locally by re-scoring word-dropout perturbations through the whole pipeline; exact TreeSHAP explains the XGBoost component locally and globally. Training is CPU-only and completes in about two minutes. The block diagram of this alternative is shown in Figure 3.1.

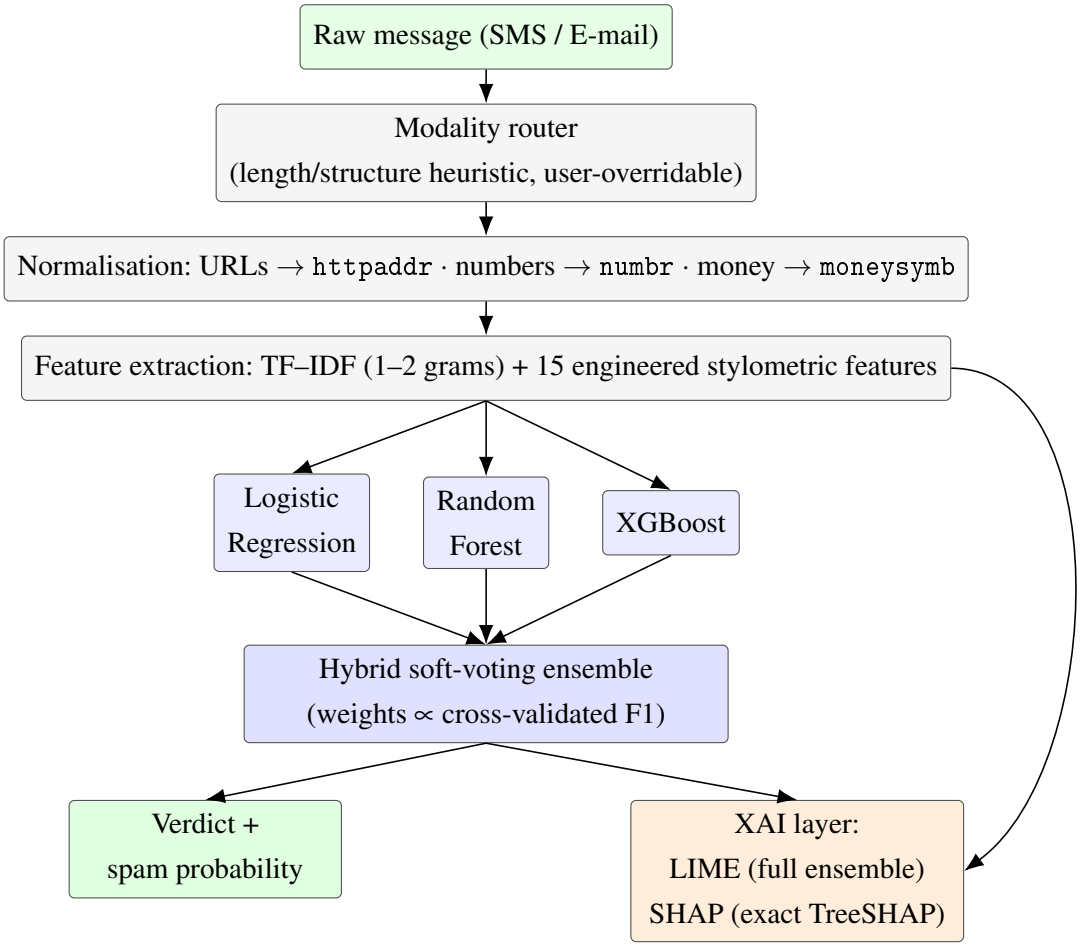


Figure 3.1: Block diagram of the proposed interpretable multi-modal detector (Design II)

3.5. Design selection

The two alternatives were scored against the constraint set, as shown in Table 3.1. Design II was selected: it reaches the accuracy band of the transformer on these corpora while satisfying every constraint the transformer violates, and its tree-based component admits *exact* Shapley-value explanations rather than approximations.

Table 3.1: Comparison and selection of the design alternatives

Criterion	**Design I (transformer)**	**Design II (hybrid + XAI)**
Expected / measured accuracy	98–99% (reported)	98.25–98.55% (measured)
Explanation fidelity	Low (attention is unfaithful)	High (model-agnostic LIME; exact TreeSHAP)
Training hardware / time	GPU; tens of minutes–hours	CPU; ≈2 minutes
Reproducibility for examiners	Moderate	High (single command + tests)
Meets full constraint set	No	Yes

3.6. Implementation plan/methodology

The selected design is realised by the pipeline of Figure 3.2. Text is vectorised with sublinear TF–IDF,

$$w_{t,d} \;=\; \big(1+\log \mathrm{tf}_{t,d}\big)\cdot \log \frac{N}{\mathrm{df}_t}, \tag{3.1}$$

and the fifteen engineered statistics are appended to the vector. Each base learner is trained with imbalance handling, and its 5-fold cross-validated F1 on the training split determines its voice in the ensemble:

$$p(\text{spam}\mid x) \;=\; \sum_{m=1}^{3} w_m\, p_m(\text{spam}\mid x), \qquad w_m \;=\; \frac{F1_m^{\mathrm{CV}}}{\sum_{k=1}^{3} F1_k^{\mathrm{CV}}}, \tag{3.2}$$

so a stronger learner gets a proportionally louder vote. The held-out test split is touched exactly once, for the final evaluation of Chapter 4.

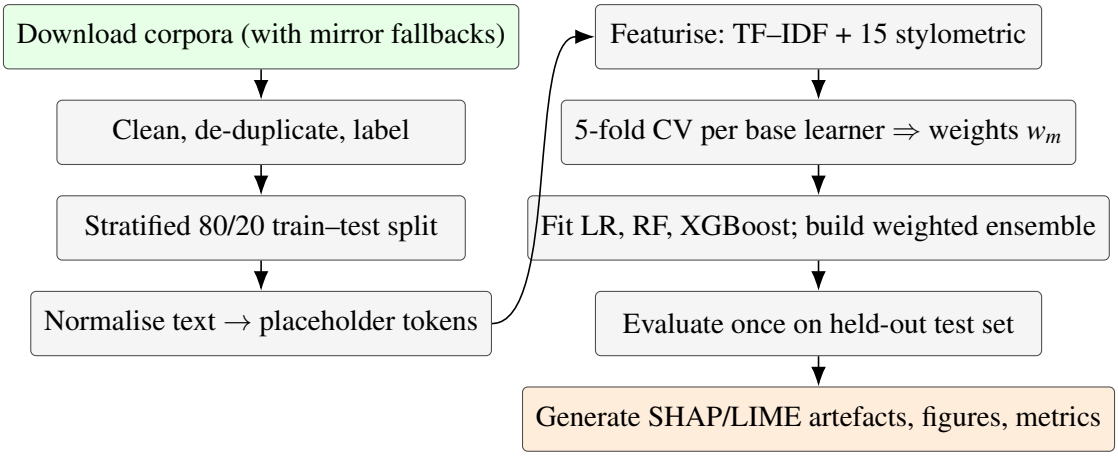


Figure 3.2: Implementation flowchart of the training and evaluation pipeline

CHAPTER 4.

RESULTS ANALYSIS AND VALIDATION

4.1. Implementation of solution

Modern engineering tools were used throughout:

- **Analysis:** Python (pandas, NumPy) and a Jupyter research notebook for exploratory data analysis and sampling studies.
- **Design and implementation:** scikit-learn (Logistic Regression, Random Forest, TF–IDF), XGBoost, and the SHAP and LIME libraries, organised as a modular `src/` package.
- **Report preparation:** LaTeX for this document; matplotlib with a shared publication style for all generated figures.
- **Project management and communication:** Git for version control and a shared Kanban board for task tracking.
- **Testing and validation:** a 13-check automated end-to-end smoke-test suite (including a simulated application session) and a strict held-out evaluation protocol.

4.1.1. Experimental setup

Two public research corpora were used; their statistics after cleaning and de-duplication are given in Table 4.1 and visualised in Figure 4.1. Each corpus received an 80/20 stratified train–test split; base-learner weights were computed by 5-fold cross-validation on the training split only, and the test split was used exactly once. Performance is reported with accuracy, precision $P = \mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$, recall $R = \mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$, their harmonic mean F1, and ROC-AUC.

Table 4.1: Corpora used for training and evaluation (after cleaning)

Modality	Corpus	Messages	Spam share	Median words
SMS	UCI SMS Spam Collection (Almeida & Gómez Hidalgo, 2011)	5,158	12.4%	12
E-mail	Enron-Spam, stratified subsample (Metsis et al., 2006)	12,000	48.3%	155

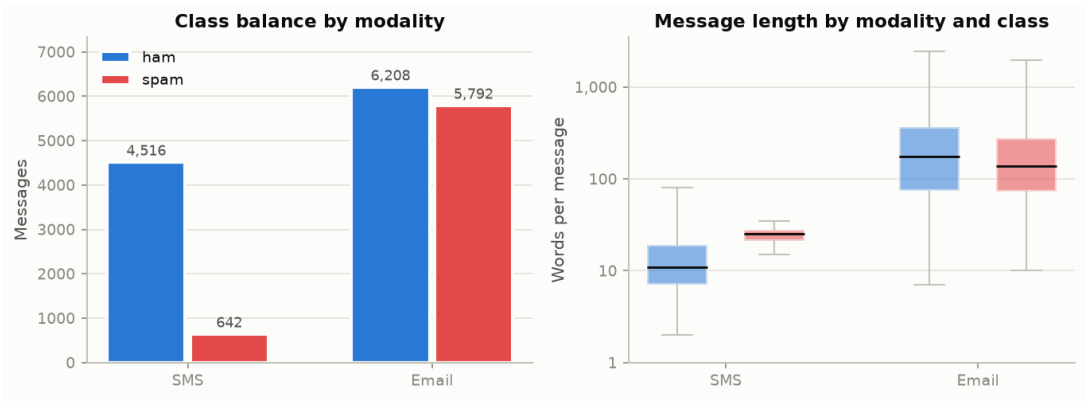


Figure 4.1: Class balance per modality and message-length distributions

The two modalities differ radically in both imbalance (12% versus 48% spam) and length (12 versus 155 median words), which is precisely why each is served by its own feature space and its own trained ensemble.

4.1.2. Classification results

Tables 4.2 and 4.3 report the complete held-out results for all four systems on each modality; Figure 4.2 compares them graphically and Figure 4.3 shows the ROC curves.

Table 4.2: SMS — held-out test set (1,032 messages)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV F1
Logistic Regression	0.9855	0.9520	0.9297	0.9407	0.9941	0.9554 ± 0.020
Random Forest	0.9845	0.9516	0.9219	0.9365	0.9929	0.9558 ± 0.017
XGBoost	0.9845	0.9667	0.9062	0.9355	0.9941	0.9586 ± 0.015
Hybrid Ensemble	**0.9855**	**0.9669**	**0.9141**	**0.9398**	**0.9941**	—

Table 4.3: E-mail — held-out test set (2,400 messages)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV F1
Logistic Regression	0.9833	0.9753	0.9905	0.9829	0.9979	0.9797 ± 0.004
Random Forest	0.9746	0.9629	0.9853	0.9740	0.9973	0.9669 ± 0.004
XGBoost	0.9750	0.9645	0.9845	0.9744	0.9969	0.9732 ± 0.003
Hybrid Ensemble	**0.9825**	**0.9705**	**0.9940**	**0.9821**	**0.9983**	—

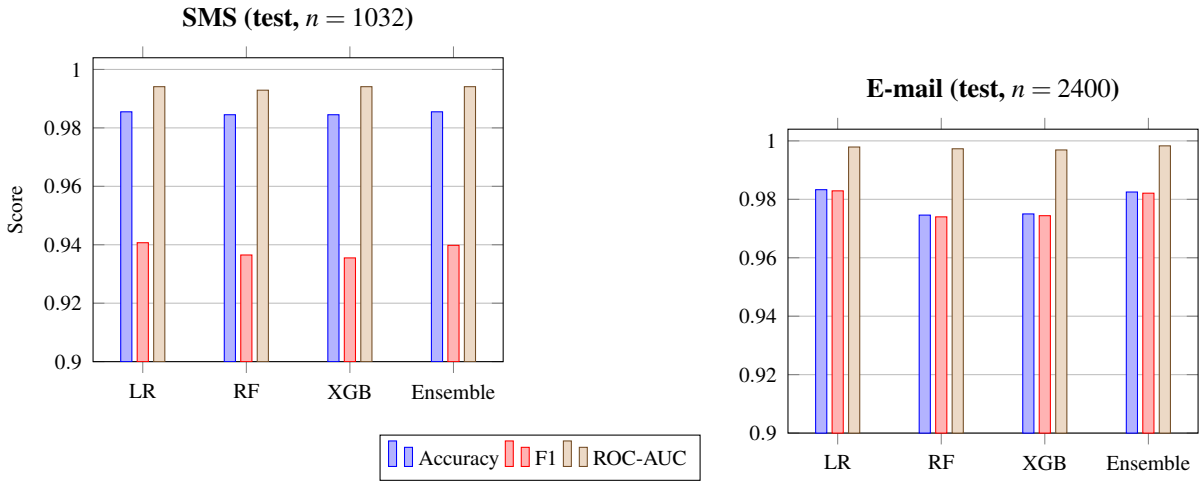


Figure 4.2: Test-set metrics per model and modality (vertical axis truncated at 0.90 to resolve the differences)

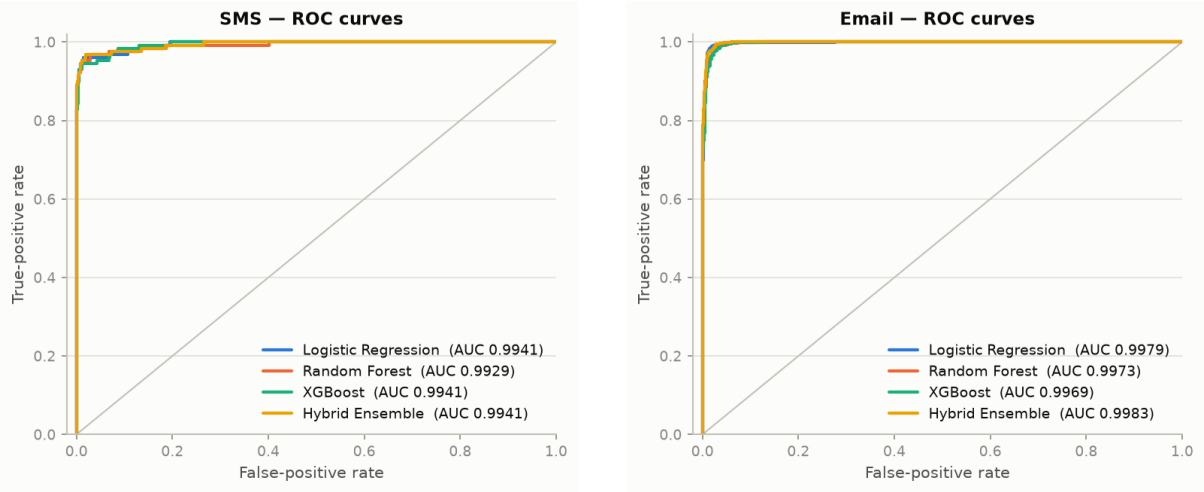


Figure 4.3: ROC curves for all four systems: SMS (left) and e-mail (right)

On ROC-AUC the hybrid ensemble ties the best base learners on SMS (0.9941, identical to four decimal places) and wins outright on e-mail (0.9983), where it also posts near-perfect recall (0.9940) at 0.9705 precision. On SMS it pairs the highest precision of any system (0.9669) with the joint-highest accuracy (0.9855, tied with Logistic Regression) — exactly the profile the user-safety constraint asked for, since on that channel a false alarm means a suppressed legitimate message.

4.1.3. Ensemble behaviour and error analysis

Figure 4.4 shows the ensemble confusion matrices derived from the held-out evaluations. The SMS ensemble mislabels only 4 of 904 legitimate messages; the e-mail ensemble misses just 7 of 1,158 spam messages. The two matrices are deliberately asymmetric: the SMS specialist errs on the side of precision, the e-mail specialist on the side of recall, matching the relative

costs of a wrong decision on each channel.

SMS ensemble ($n = 1032$)

	Pred. ham	Pred. spam
Actual ham	TN **900**	FP **4**
Actual spam	FN **11**	TP **117**

E-mail ensemble ($n = 2400$)

	Pred. ham	Pred. spam
Actual ham	TN **1207**	FP **35**
Actual spam	FN **7**	TP **1151**

Figure 4.4: Confusion matrices of the hybrid ensembles on the held-out test sets

4.1.4. Explainability results

4.1.4.1. A verdict, explained (local scope)

The following scam SMS, never seen in training, is used as a live exhibit:

> "URGENT! You have WON a £2,000 cash prize GUARANTEED. Call 09061701461 now or visit www.claim-your-prize.com to claim!"

All three base learners vote spam — Logistic Regression 1.000, Random Forest 0.987, XGBoost 1.000 — giving a weighted ensemble probability of **0.996 spam**. LIME re-scores approximately 1,000 word-dropout perturbations of the message through the entire pipeline and fits a local linear surrogate; the resulting per-word contributions are plotted in Figure 4.5 alongside the SHAP waterfall of the same prediction. The premium-rate phone number is the single strongest signal, followed by the money amount and the classic scam vocabulary — a justification a non-expert can check at a glance.

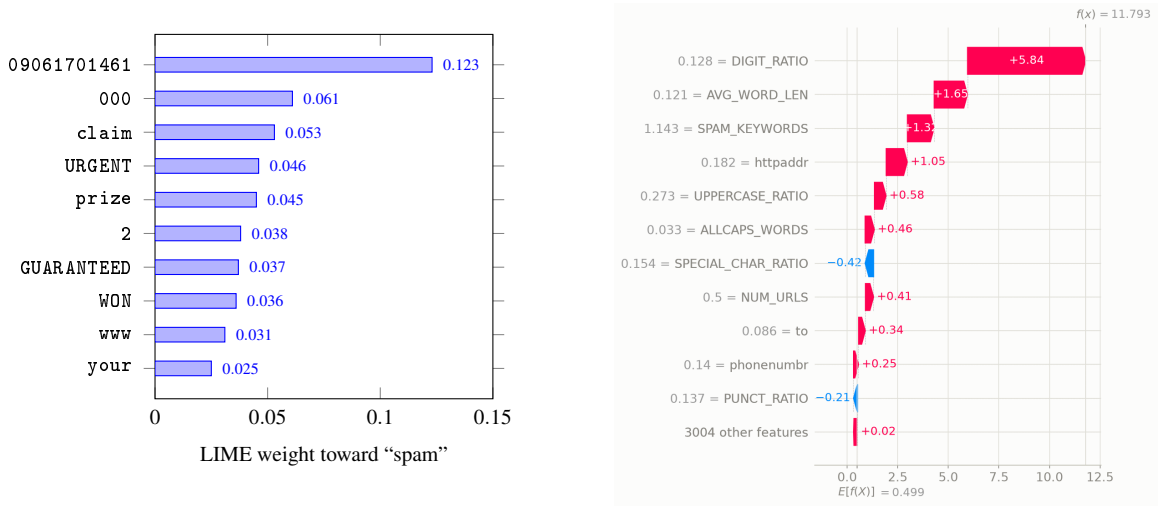


Figure 4.5: Local explanations of the exhibited SMS: LIME word attributions over the full ensemble (left) and SHAP waterfall of the XGBoost component (right)

4.1.4.2. What the models learned (global scope)

Global SHAP summaries were computed over 800 test messages per modality (Figure 4.6). The key finding is that *stylometry rivals vocabulary*: on SMS the five top-ranked features are all engineered stylometric statistics — `DIGIT_RATIO`, `AVG_WORD_LEN`, `SPAM_KEYWORDS`, `UPPERCASE_RATIO` and `CHAR_COUNT` — and the fifteen engineered features together carry about three-quarters (74.5%) of the total attribution, although they are only 15 of the 3,015 features. A scam is betrayed by its *shape* as much as by its words.

All SHAP values in this chapter are computed with XGBoost's native, exact TreeSHAP on the same sparse feature matrix that the model scores. This matters: XGBoost treats entries that are absent from a sparse matrix as missing values, so explaining a densified copy of the input would describe a different function — in this pipeline, densification changes the XGBoost verdict for 729 of the 1,032 SMS test messages.

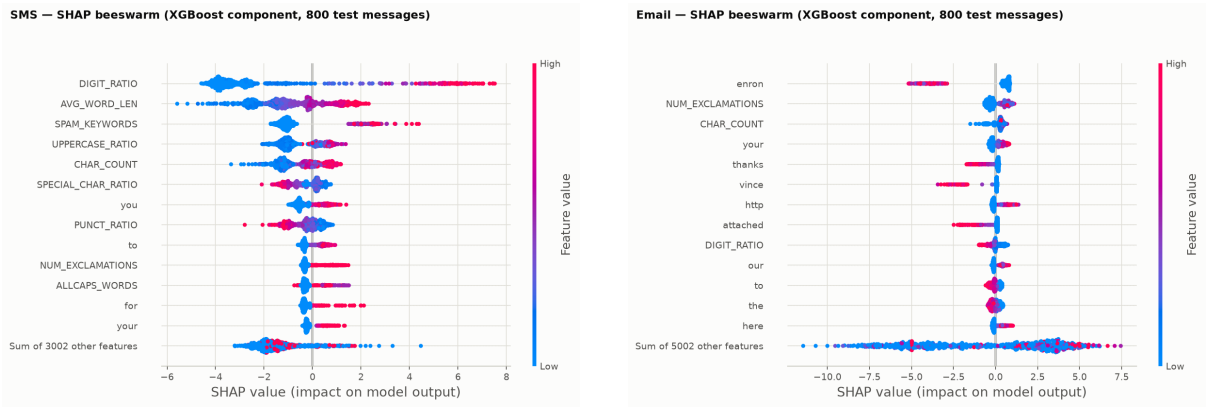


Figure 4.6: Global SHAP beeswarm summaries: SMS (left) and e-mail (right); UPPER-CASE rows are engineered stylometric features, lower-case rows are words

4.1.4.3. XAI as a debugging tool: a dataset bias, caught

On the e-mail model, the global SHAP ranking exposed the token `enron` as a strong *ham* indicator: the corpus's legitimate mail is all Enron-internal, so the model partly learned the dataset rather than the concept. This bias is invisible in every accuracy table in this chapter and obvious in one beeswarm plot — the clearest argument this project makes for shipping explanations together with predictions.

4.1.5. Interpretation and data validation

Cross-validation dispersion is small (F1 standard deviation between ± 0.003 and ± 0.020 across all six base learners), indicating stable models rather than lucky splits, and the derived ensemble weights are near-uniform (Appendix-3), so the ensemble's gains come from the *diversity* of the three learners rather than from one dominant model. The 13-check automated smoke-test suite

exercises the entire system — data loading, preprocessing, both detectors, LIME, SHAP and a simulated application session — and passes end-to-end; the complete pipeline reproduces from a clean checkout in about three minutes on a commodity CPU, with training itself taking about two minutes. The dataset downloader validates its sources and falls back to mirrors automatically, and the modality router can be overridden by the user in the application, so a misrouted message can always be re-scored by the other specialist.

CHAPTER 5.

CONCLUSION AND FUTURE WORK

5.1. Conclusion

Every objective set in Section 2.6 was met. The hybrid ensemble reached 98.55% accuracy on SMS and 98.25% on e-mail against a target of 97%, and it matched (to four decimal places) or beat every base learner on ROC-AUC on both modalities (0.9941 on SMS; an outright best 0.9983 on e-mail). Every single prediction is delivered with a local LIME explanation over the full ensemble and a SHAP attribution of the boosted-tree component, and global beeswarm and importance summaries describe what each model has learned. The complete pipeline — corpus download, training, evaluation and figure generation — reproduces from a clean checkout in about three minutes on a commodity CPU and is verified by a 13-check automated test suite, and the four-tab Streamlit application makes the system operable by a non-expert.

The most instructive result, however, was not a number. The global SHAP analysis exposed the token `enron` acting as a shortcut for legitimate mail — a genuine dataset bias in the e-mail corpus that deviated from the naive expectation that a 98%-accurate model has learned the concept of "spam". The bias is a property of the data, not a fault of the pipeline, but it would have shipped silently inside any black-box deployment. Catching it required the explanation layer, which is precisely the thesis this project set out to demonstrate: interpretability is not a presentation feature bolted onto a classifier, it is a debugging and auditing instrument that changes what the engineer can see.

5.2. Future work

- Extend the framework to multilingual and code-mixed corpora, since the present datasets are English-only and date from 2002–2012.
- Perform cross-domain evaluation (train on one corpus, test on another) to quantify and then mitigate the dataset bias exposed by SHAP.
- Add probability calibration so that the reported spam probabilities can be thresholded meaningfully by downstream policy.
- Harden the models against adversarially obfuscated spelling (e.g. leet-speak substitutions) and measure robustness explicitly.
- Add counterfactual explanations — "what is the smallest edit that flips this verdict?" — as a third, complementary XAI technique.
- Benchmark a fine-tuned transformer under the same explanation harness to quantify the accuracy–interpretability trade-off empirically.

REFERENCES

1. Almeida, T.A. and Gómez Hidalgo, J.M. (2011) 'SMS Spam Collection v.1', UCI Machine Learning Repository.
2. Breiman, L. (2001) 'Random Forests', Machine Learning, Vol. 45, No. 1, pp. 5–32.
3. Chen, T. and Guestrin, C. (2016) 'XGBoost: A Scalable Tree Boosting System', Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794.
4. Cranor, L.F. and LaMacchia, B.A. (1998) 'Spam!', Communications of the ACM, Vol. 41, No. 8, pp. 74–83.
5. Drucker, H., Wu, D. and Vapnik, V.N. (1999) 'Support Vector Machines for Spam Categorization', IEEE Transactions on Neural Networks, Vol. 10, No. 5, pp. 1048–1054.
6. Federal Bureau of Investigation (2024) 'Internet Crime Report 2023', Internet Crime Complaint Center (IC3), Washington, DC.
7. Lundberg, S.M. and Lee, S.-I. (2017) 'A Unified Approach to Interpreting Model Predictions', Advances in Neural Information Processing Systems 30, pp. 4765–4774.
8. Metsis, V., Androutsopoulos, I. and Paliouras, G. (2006) 'Spam Filtering with Naive Bayes — Which Naive Bayes?', Proc. 3rd Conference on Email and Anti-Spam (CEAS), Mountain View, CA.
9. Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', Journal of Machine Learning Research, Vol. 12, pp. 2825–2830.
10. Ribeiro, M.T., Singh, S. and Guestrin, C. (2016) '"Why Should I Trust You?": Explaining the Predictions of Any Classifier', Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1135–1144.
11. Sahami, M., Dumais, S., Heckerman, D. and Horvitz, E. (1998) 'A Bayesian Approach to Filtering Junk E-mail', AAAI Workshop on Learning for Text Categorization, Madison, WI.
12. Telecom Regulatory Authority of India (2018) 'Telecom Commercial Communications Customer Preference Regulations', New Delhi.

APPENDIX

Appendix-1: Software environment

S. No.	Component	Role in the framework
1	Python 3.10–3.12	Implementation language of the entire pipeline
2	scikit-learn	Logistic Regression, Random Forest, TF–IDF, metrics, cross-validation
3	XGBoost	Gradient-boosted tree base learner
4	SHAP	Exact TreeSHAP local and global attributions
5	LIME	Model-agnostic local explanations of the full ensemble
6	pandas / NumPy	Data handling and numerical computation
7	matplotlib	All generated figures (shared publication style)
8	Streamlit	Four-tab interactive demonstration application
9	Git	Version control and collaboration

Appendix-2: Engineered stylometric features (representative)

S. No.	Feature	Description
1	`DIGIT_RATIO`	Fraction of characters that are digits
2	`UPPERCASE_RATIO`	Fraction of characters in upper case
3	`CHAR_COUNT`	Message length in characters
4	`AVG_WORD_LEN`	Average word length
5	`NUM_EXCLAMATIONS`	Count of exclamation marks
6	`NUM_URLS`	Number of URLs detected
7	`NUM_PHONES`	Number of phone numbers detected
8	`SPAM_KEYWORDS`	Hits against the transparent scam-keyword lexicon

The full set of fifteen features, together with the complete keyword lexicon, is defined in plain sight in `src/config.py` and `src/features.py` so that it can be audited without reading model internals.

Appendix-3: Derived ensemble weights (normalised CV F1)

Modality	**Base learner**	**CV F1**	**Weight w_m**	
SMS	Logistic Regression	0.9554	0.3329	$\sum w_m = 1$
	Random Forest	0.9558	0.3331	
	XGBoost	0.9586	0.3340	
E-mail	Logistic Regression	0.9797	0.3356	$\sum w_m = 1$
	Random Forest	0.9669	0.3311	
	XGBoost	0.9732	0.3333	

The near-uniform weights show that the three learners are comparably strong; the ensemble's advantage therefore comes from their diversity (linear / bagging / boosting) rather than from any single dominant model.

USER MANUAL

Prerequisites. Python 3.10–3.12, about 2 GB of free disk space, and an internet connection for the first run (the two public corpora are downloaded automatically, with mirror fallbacks).

1. Obtain the project source code and open a terminal in the project root directory.
2. Create and activate a virtual environment:
   `python -m venv .venv && source .venv/bin/activate`
   (on Windows: `.venv\Scripts\activate`).
3. Install the pinned dependencies:
   `pip install -r requirements.txt`
4. Build the datasets and train both detectors (about two minutes; downloads the corpora on the first run):
   `python -m src.data_loader`
   `python -m src.train`
5. Verify the installation end-to-end (13 automated checks, including a simulated application session):
   `python tests/smoke_test.py`
6. Launch the demonstration application:
   `streamlit run app.py` then open `http://localhost:8501` in a browser.
7. In the **Detector** tab, paste any message (or load a real held-out example) and press *Analyse*. The verdict card shows the ensemble spam probability, each base model's vote with its weight, the message with LIME-highlighted words, the SHAP waterfall and the engineered feature values (Figure 5.1).
8. Use the **Model performance** tab for the metric tables, ROC curves and confusion matrices; the **Global explanations** tab for the interactive beeswarm and importance analyses; and the **About** tab for the architecture and references.
9. If the modality router picks the wrong specialist for an unusual message, override it manually with the SMS/E-mail selector and re-analyse. To rebuild everything from scratch, run `python -m src.data_loader -force` and repeat step 4.

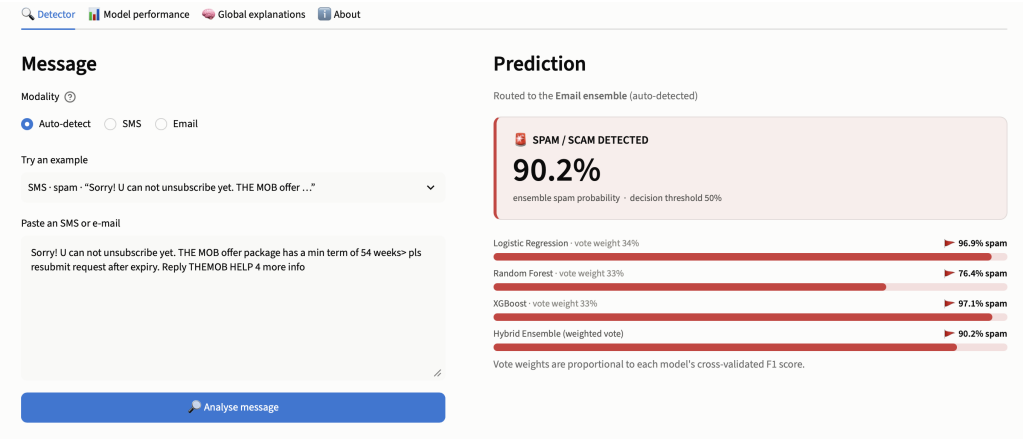


Figure 5.1: Detector tab of the application after analysing a message